

A light-weight, compact dynamic array

Thomas Köppe <tkoeppe@google.com>

Contents

1. [Summary](#)
2. [Impact](#)
3. [Synopsis](#)
4. [Limitations](#)
5. [Appendix I: What's wrong with dynamic arrays?](#)
6. [Appendix II: Early feedback](#)
7. [Acknowledgements](#)

Summary

Short form. We propose a light-weight, dynamic array which is in every aspect identical to `std::vector<T, A>` except that it does not allow efficient size-changing operations. For many use cases, the two containers can be used interchangeably, but the light-weight container will typically only consume 2/3 of the space of a vector, since it does not need to keep track of a capacity separately from the size. In other words, it is just like `std::vector<T, A>` with the following modifications:

- No `insert`, `push_back`, `emplace{, _back}`, `erase`.
- No `reserve`, `capacity`.
- `resize(n)` always reallocates unless `n == size()`.

Bike shed: what is it called? The first obvious problem is how to name this container. Names like `flexible_array` or `flarray` seem unwieldy, `small_vector` suggests SSO-like optimisations, `vlarray` is too much like VLA or `valarray`, and `dyn_array` is too close to `dynarray`. As a placeholder we use “`compact_vector`” for now, with the understanding that this will need to be revised. We are open to suggestions.

What this proposal is not: A preliminary discussion on the reflector showed that a lot of people have specific expectations, desires and goals in this design space. We would like to establish clear boundaries for this proposal:

- Not a proposal for a fixed-size array. The proposed container can be resized.
- Not a proposal for an unmovable array. The proposed container can be moved and swapped as usual.
- Not a proposal for “stack-allocation optimisation”. The proposed container is an ordinary, dynamic container that uses an allocator to obtain storage.
- Not a fix or revival of `std::dynarray`. This proposal does not share the goals of `dynarray`, and it comes with its own motivation and rationale.

First and foremost we would like this proposal to be considered in isolation and on its own merits. We will gladly work on unification and harmonising with the authors of any other paper in this design space that LEWG finds desirable, once that time comes.

Long rationale. The author considers dynamic arrays (i.e. `new T[n]()`) a misfeature of the language (see the [appendix](#) for a summary of its failings). In almost every way, one may replace such a dynamic array with `std::vector<T>(n)`, except for the undeniable fact that a vector is larger. While we already need to store and communicate the size of a dynamic array in order to use it safely, there is generally no reason for the cost of keeping a separate capacity counter around. Until now, the library has been missing a simpler dynamic container that meets precisely, and without the cost of unwanted generality, those needs which currently call for dynamic arrays.

The solution. We propose a new container class template, for now and for exposition only referred to as `compact_vector`, which is in every aspect like `std::vector`, except that it does not have growing insertion

operations, shrinking erasure operations, or a notion of capacity. Note in particular that we still have `resize()`.

Example.

```
compact_vector<compact_vector<short>> v(1000000);

for (auto & x : v) {
    x.resize(make_number(), make_default_value());
    process(x.begin(), x.end());
}
```

Using a `vector<short>` instead of `compact_vector<short>` would require a million unused additional capacity pointers. If the size of each `v[i]` is small, this may well be a non-trivial cost which the proposed container avoids.

Impact

This is a pure library extension.

Synopsis

```
template <typename T, typename Alloc = allocator<T>>
class compact_vector {
public:
    // "types" as in vector

    // "constructor/copy/destroy" as in vector, requiring ForwardIterator

    // "iterators" as in vector

    // "capacity"
    size_type size() const noexcept;
    bool empty() const noexcept;
    void resize(size_type n);
    void resize(size_type n, const T & value);

    // "element access" as in vector

    // "data access" as in vector

    // "modifiers"
    void swap(compact_vector &) noexcept(allocator_traits::
<A>::propagate_on_container_swap::value || allocator_traits::<A>::is_always_equal::value);
    void clear() noexcept;

    // relational operators as in vector

    // "specialized algorithms" as in vector

    explicit operator vector<T, Alloc>&&() && noexcept;
};
```

Notes

- The two-iterator constructor and the two-iterator overload of `assign` require forward iterators (so as to be able to precompute the size).

- The explicit conversion to vector is a stealing conversion; we expect `compact_vector` to be implemented in such a fashion that the allocated storage can be transplanted into a `vector` with the same template arguments. Note that we do *not* have the opposite operation, stealing *from* a vector. To do so we would require a vector whose size is equal to its capacity, which cannot be achieved portably. This would be nice to have in the future, since it would allow using a `vector` as a “builder” for a “frozen” `compact_vector`.
- Iterators and references remain valid unless a `resize` function is called with `n != size()`.

Limitations

We can replace uses of `new T[n]()` and most uses of `new T[n]` with `compact_vector<T>(n)`. The only feature that we deliberately choose not to provide is uninitialised storage, like `new char[n]`. User code benefits from uninitialised allocations (and this is a real use case!), but it requires cooperation from the user for correctness. Users requiring uninitialised storage can resort to `std::unique_ptr<char[]>(new char[n])`.

Appendix I: What’s wrong with dynamic arrays?

Dynamic arrays, i.e. arrays created via either `new T[n]` or some placement variant thereof, are an odd-man-out in many regards. We will enumerate a few.

- Dynamic arrays cannot be used independently of their value without separately communicating their size, which needs to be captured somewhere else explicitly at the time of construction of the array. Yet the size also needs to be stored internally in the implementation to allow for destructor calls. The former violates encapsulation, the latter economy.
- Dynamic arrays are the *only* part of C++ in which truly dynamic typing happens: An object is created whose type is not known statically, and there is not even an expression whose value is that object. This is weird. (By contrast, `Base * p = new Derived` does at least contain an expression with which one might statically derive a description of the created object, even though that code snippet does not capture it.)
- Dynamic arrays cannot be constructed and destroyed like normal objects: `new T` means something surprising when `T` is an array type, and `p->~T()` is ill-formed. This means, for example, that there is no straightforward notion of an `allocate_shared` for a shared pointer to an array; rather, we have a sequence of elements under shared ownership which are destroyed by a hand-crafted deleter that destroys array *elements*.
- Dynamic arrays have repeatedly been the subject of arcane standard defects. The inability to use array-placement-new on account of unknowable overhead (“+ y”) remains unresolved (cf. [CWG 476](#), [EWG 68](#)).
- Dynamic arrays generally conflate the two responsibilities of memory allocation and object construction. The library’s pervasive use of allocators to create memory and objects separately, one-by-one under the control of the user, has been demonstrated as the superior way to manage collections of objects. Hundreds of student hours spent on implementing Baby’s First Vector by starting with `T * ptr = new T[n];` have shown the general confusion between something that is empty and something which consists of `n` default-constructed objects.

Appendix II: Early feedback

A draft of this paper was circulated on the LEWG reflector for early feedback. Here are some arbitrarily selected responses.

- Many people are indifferent towards this proposal. There is a trade-off between adding more library types and the benefits of having a perfect fit for this particular use case.
- Herb Sutter suggested that the C++ Core Guidelines are [looking for a type like this](#), for the same reasons: To replace array-new.
- Several people suggested that their users are interested in stack optimisations, not this.
- There were suggestions that SG14 may be interested.

Acknowledgements

The author wishes to thank everyone who provided valuable feedback on the LEWG reflector. Particular thanks go to David Krauss, Edward Smith-Rowland and Howard Hinnant for improving the stealing interface, and to Ville Voutilainen for suggesting the name `compact_vector`.

